PROJETO PRÁTICO · ACOMPANHAMENTO DO ESTUDANTE

Da ideia à solução com IA

Checklist de desenvolvimento com plataformas digitais e Vibecode / IA. Marque apenas o que já foi realizado e registre as evidências de cada etapa.

1. Definir o desafio → 2. Escolher a plataforma → 3. Integrar IA → 4. Testar e validar → 5. Documentar

Nomes dos estudantes (mínimo 2 e máximo 3)

Estudante 1 (obrigatório)

Diego Henrique Silva Soares

Estudante 2 (obrigatório)

Luan Bispo Silva

Estudante 3 (opcional)

Turma / disciplina

Tendência em Ciência da Computação | N1 2122

Nome do projeto

Central de Dúvidas com Triagem Inteligente

Data do preenchimento

15/10/2026

01

08/10

Definir o desafio

4 / 4

Escolha um problema simples e com escopo bem definido.

- ☑ Descrevi o problema e o público que será beneficiado.
- ☑ Defini o objetivo e o resultado esperado.
- ☑ Delimitei o que a solução fará e o que ficará fora do escopo.
- ☑ Estabeleci critérios para verificar se o desafio foi resolvido.

Problema, público, objetivo e critérios de sucesso

Problema: dúvidas recorrentes dos alunos (prazos, avaliação, acesso a sistemas, taxas) chegam de forma desorganizada, são respondidas manualmente e não ficam registradas.
Público: alunos da disciplina e a equipe que responde às dúvidas.
Objetivo: criar uma central que receba dúvidas por formulário, classifique cada uma por categoria e urgência com IA, responda automaticamente a partir de uma FAQ e registre tudo em uma planilha.
Fora do escopo: login de alunos, atendimento humano dentro da ferramenta e histórico de conversas.
Critérios de sucesso: (1) dados inválidos são barrados; (2) dúvidas presentes na FAQ recebem resposta correta por e-mail; (3) dúvidas fora da FAQ não recebem resposta inventada e vão para revisão manual; (4) falhas da IA são registradas sem perder o chamado.

02

15/10

Escolher e configurar a plataforma

4 / 4

Selecione a ferramenta adequada ao caso de uso: Bubble, Make, Zapier, Airtable, Pipefy, Adalo ou outra.

- ☑ Comparei as ferramentas considerando o caso de uso.
- ☑ Verifiquei limites de uso, custos e recursos necessários.
- ☑ Escolhi a plataforma e justifiquei minha decisão.
- ☑ Configurei o ambiente e executei um teste inicial.

Plataforma escolhida, justificativa e configuração realizada

Plataformas: Google Forms (entrada), Google Sheets (registro e painel), Make (automação), API do Gemini (IA) e Gmail (envio).
Justificativa: a restrição do projeto era custo zero e sem consumir períodos de teste. O Zapier foi a primeira opção, mas o plano gratuito limita os fluxos a 2 etapas e não tem filtros nem desvios; o nosso fluxo precisava de 6 ou mais. O Make gratuito permite várias etapas, roteadores, filtros e tratamento de erro. O Gemini foi escolhido por ter free tier, já que o módulo OpenAI do Make exige chave paga.
Configuração: formulário com validação, planilha vinculada com as colunas Categoria, Urgência, Resposta e Status, cenário no Make com gatilho de novas linhas. O teste inicial confirmou que as respostas do formulário chegavam à planilha e eram lidas pelo Make.

03

22/10

Integrar Vibecode / IA

5 / 5

Dialogue com a IA de forma iterativa para estruturar configurações, lógicas leves ou integrações.

- ☑ Escrevi um prompt com objetivo, contexto e restrições.
- ☑ Revisei a resposta da IA e solicitei ajustes quando necessário.
- ☑ Implementei uma lógica, configuração ou integração compatível com o desafio.
- ☑ Registrei os prompts, as alterações e os resultados obtidos.
- ☑ Mantive senhas, chaves de API e dados pessoais fora dos prompts e do código público.

Prompts utilizados, integração implementada e ajustes realizados

Prompts: duas versões registradas no repositório (prompts/v1.md e v2.md). A v1 definia tarefa, regras contra injeção de instruções, proibição de inventar e saída em JSON, mas a FAQ estava vazia. A v2 incluiu a FAQ, critérios de categoria e urgência, separação entre “Outro” e “Revisão manual”, proteção do prompt e tratamento de várias perguntas.
Integração: chamada HTTP à API do Gemini (modelo Flash-Lite), com corpo montado pelo módulo Create JSON (escape seguro do texto), resposta em JSON lida pelo Parse JSON, envio por Gmail e gravação na planilha.
Ajustes: correção do ID do modelo e do campo responseMimeType (erro 400); troca de modelo após erros 503 recorrentes; validação da saída da IA com switch() e ifempty(); chave da API guardada em credencial do Make restrita ao domínio do Google.
Resultado: a pergunta “Qual a capital da França?” foi classificada como Revisão manual na v1 e corretamente como Outro na v2.

04

29/10

Testar e verificar a segurança

6 / 6

Teste toda lógica gerada por IA. Valide entradas e trate cenários de erro.

- ☑ Testei o fluxo principal com dados fictícios.
- ☑ Testei campos vazios, dados inválidos e entradas inesperadas.
- ☑ Verifiquei mensagens de erro e falhas de integração.
- ☑ Conferi permissões de acesso e proteção de credenciais.
- ☑ Corrigi problemas e repeti os testes.
- ☑ Comparei os resultados com os critérios de sucesso.

Casos de teste, resultados, falhas e correções

Casos de teste: pergunta presente na FAQ (prazo do trabalho), urgência alta (AVA + prazo amanhã), fora do escopo (capital da França), tentativa de prompt injection (“ignore as regras… nota 10”), dúvida com aspas, várias perguntas na mesma mensagem, e-mail inválido, dúvida com menos de 10 caracteres e erro proposital de API (modelo inexistente).
Resultados: respostas corretas a partir da FAQ, urgência alta identificada, injeção ignorada, inválidos barrados no formulário e no Make, e falhas da API registradas como Status Erro sem envio de e-mail.
Falhas e correções: validação do formulário mal configurada (dúvida “3 L” passou); filtro com valor vazio reprovando tudo; router sem rota fallback marcando válidos como inválidos; resposta gravada na linha errada; erros 400 e 503 da API; e-mails de teste enviados a endereços de terceiros, trocados por aliases. Todos foram corrigidos e retestados.
Comparação com os critérios: os quatro critérios foram atendidos. Ficaram pendentes o teste com quebra de linha real e o de vazamento do prompt, porque ambos caíram em erro 503.

05

Consolidação final

Documentar a solução

5 / 5

Consolide os registros do projeto e as instruções para utilização. Prazo conforme orientação docente.

- ☑ Descrevi o problema, a solução e a plataforma utilizada.
- ☑ Registrei como configurar e utilizar a solução.
- ☑ Incluí evidências dos testes e das melhorias.
- ☑ Registrei limitações, referências e uso de IA.
- ☑ Revisei os registros antes de imprimir ou salvar em PDF.

Link do projeto, evidências, limitações e próximos passos

Link do projeto: https://github.com/SoaresDiego8/Tendencia-Em-Ciencia-da-Computacao/tree/main/Unidade3/Projeto-Low-Code
Evidências: prints do formulário, do cenário no Make, da planilha com os resultados (e-mails ocultados), do e-mail de resposta e dos erros 400 e 503, além dos slides em PDF.
Limitações: dependência de serviços gratuitos (503 em 3 de 12 chamadas e possível uso dos dados pelo provedor); lógica visual com regras ocultas (router não exclusivo, erros silenciosos); IA não determinística e aprisionamento à plataforma.
Próximos passos: retentativa automática para 503, envio por conta institucional, FAQ editável numa aba da planilha, alerta para urgência alta e conclusão dos testes pendentes.

Etapas do desenvolvimento · Checklist da equipe · As datas seguem o cronograma informado; a documentação não possui prazo definido neste roteiro.

```
```